

2026 암호분석경진대회

8번 문제

1) 제공된 데이터 파일과 노출된 중간값을 활용하여, 암호화에 사용된 16바이트 마스터키를 복원하시오.

문제의 암호는 변형된 7라운드 AES이다. 표준 AES와 비교하여 라운드 수와 키 스케줄링 방식이 다르며, 초기 AddRoundKey 단계가 생략되고 마지막 라운드에 MixColumns 연산이 포함된다는 구조적 차이가 있다.

제공된 "leaked.txt"에는 28178번째 평문의 암호화 과정에서 노출된 일부 중간 상태가 주어진다. 초기 평문인 X^0 가 주어졌을 때 SB, SR, MC 단계를 거쳐 W^0 를 구하는 과정은 별도의 정보를 요구하지 않는다. 또한 누출 정보에 1라운드의 첫 번째 출력 바이트인 $X^1[0]$ 이 주어지므로 앞서 구한 $W^1[0]$ 과 XOR 연산을 수행하여 ARK 단계에서 사용된 첫 번째 마스터키 바이트 $K[0]$ 을 확정할 수 있다. 계산 수식과 확정된 $K[0]$ 는 다음과 같다.

$$MC(SR(SB(X^0)))[0] \oplus K[0] = W^0[0] \oplus K[0] = X^1[0]$$

$$K[0]=0x29$$

확정된 $K[0]$ 를 제외한 나머지 K 는 제약 만족 문제 해결 엔진인 CP-SAT Solver를 활용하여 구했다. "leaked.txt"에 있는 누출 정보는 연속되는 바이트 형태가 아니라 특정 라운드의 특정 비트들만 띄엄띄엄 노출되는 파편화된 형태를 띄고 있다. 이러한 비트 정보들을 제약 조건으로 모델링하면 CP-SAT Solver의 가지치기 메커니즘이 잘 활용할 수 있을 것이라고 생각하였고 이를 통해 문제를 해결하였다. 구체적인 CP-SAT Solver 모델링 과정은 바이트 표현, AES 함수 구현, 제약조건 추가로 총 3가지 과정으로 설명하고자한다.

첫번째 바이트 표현 과정에서는 각 바이트를 다음 두 표현으로 동시에 관리하였다.

1. 0부터 255까지 정수 변수
2. 8개의 Boolean 비트 변수

이는 S-box와 같은 비선형 부분은 1번을 활용하여 테이블 제약으로 쉽게 표현하고 MixColumns, ShiftRow, AddRoundKey와 같은 선형 부분 2번을 활용하여 비트 XOR 제약으로 쉽게 표현하기 위해서이다.

파이썬 코드상에서 Byte 클래스를 정의하여, 미지수 바이트는 정수 변수와 8개의 비트 변수를 동시에 생성한 뒤 두 값이 항상 일치하도록 model.add()로 동기화 제약을 걸었다. 반면 이미 확정된 바이트는 고정 상수로 선언하고 그에 대응하는 비트들도 실제 값과 일치하도록 강제하였다.

두번째 AES 함수 구현 과정에서는 암호화 과정을 CP-SAT Solver에 구현하고 제약 조건으로 모델링 하였다. S-box의 경우 테이블 제약(model.AddAllowedAssignments)으로 구현하였고, ShiftRows는 슬라이싱을 이용하여 shift_rows() 함수로 구현하였다, MixColumns, AddRoundKey의 경우 비트 단위의 XOR연산(model.AddBoolXOr)을 활용하였으며, 특히 MixColumns에 필요한 갈루아 필드상의 2배수, 3배수 곱셈은 비트 시프트와 XOR 연산의 조합으로 이루어진 xtime, mul2, mul3 함수를 별도로 구현하여 행렬 연산하였다.

그 외에 문제에 제시된 규칙에 따라 라운드키 생성 함수(round_keys)를 구현하고 SubBytes부터 AddRoundKey까지의 단계를 차례대로 묶어 7라운드 전체를 수행하는 함수(encrypt)를 구현하였다.

세번째 제약조건 추가 단계에서는 "leaked.txt"에서 확인할 수 있는 평문(X^0), 암호문(X^7), 일부 알려진 중간 상태들을 제약으로 추가한다. 또한 앞서 구한 $K[0] = 0x29$ 도 고정 상수 제약으로 추가하였다. 실행 결과 설정한 타임아웃 900초 이내에 다음과 같이 key를 얻을 수 있었다.

```
status OPTIMAL
key hex = 2923be84e16cd6ae529049f1f1bbe9eb
```

그림1. 실행결과의 마지막 부분

key = 0x2923be84e16cd6ae529049f1f1bbe9eb

해당 key를 사용하여 "plaintext.bin"에 있는 50000개를 평문을 모두 암호화 한 후 "ciphertext.bin"와 비교하여 검

증한 결과 모두 통과하였다. 이를 통해 올바른 key를 구했음을 확인할 수 있다.

CP-SAT Solver를 활용한 풀이 코드는 “solve.py”, 검증 코드는 “check.py”에서 확인할 수 있다.